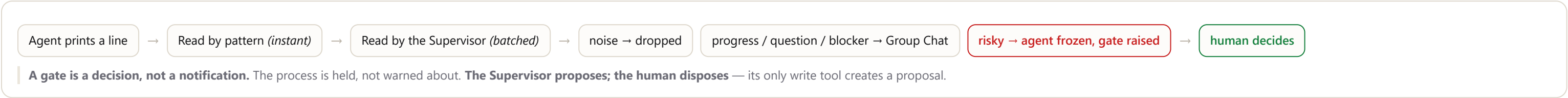


Conduit — architecture in detail

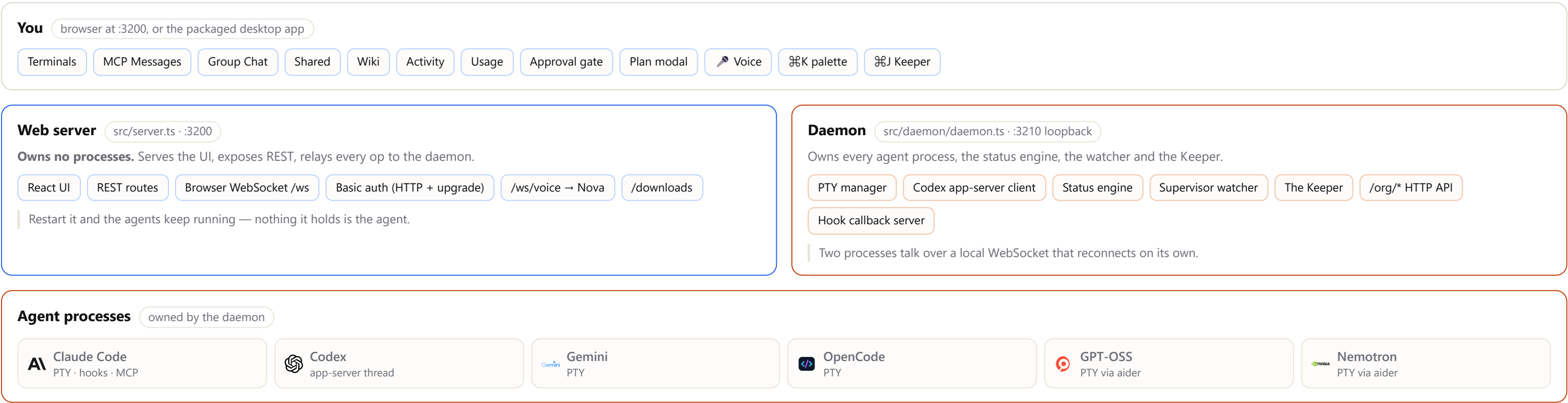
Two processes. The daemon owns every agent terminal; the web server only relays. Every line an agent prints is read twice.

Companion to ARCHITECTURE.md. Rendered from scripts/architecture2.html by npm run architecture:deep.

1 · CORE PARADIGM



2 · TOPOLOGY



How each is told about the project

CLI	CONTEXT MECHANISM	STATUS FROM	MCP
Claude Code	<code>--add-dir</code>	lifecycle hooks	✓
Codex	writable roots	app-server events	—
Gemini	<code>--include-directories</code>	process liveness	—
OpenCode	<code>AGENTS.md</code>	process liveness	—
GPT-OSS	<code>--read AGENTS.md</code>	process liveness	—
Nemotron	<code>--read AGENTS.md</code>	process liveness	—

`src/cli-registry.ts` is the single source of truth: id, binary, install command, required env. Adding a CLI is one edit.

Preflight, and two deliberate aider flags

- 1

Checked before spawn.
A missing binary fails immediately with the command that installs it, rather than opening a PTY that lands in a shell and looks alive.
- 2

`--no-auto-commits`
aider commits after every edit by default, which would slip changes past the gates and into your history.
- 3

`--yes-always` is NOT set
It would auto-approve aider's own confirmations and defeat the gates entirely. Those confirmations are what the regex path watches for.
- Output is buffered, so a browser attaching *after* an agent started gets a replay, not a blank pane. A browser attaching before it runs is remembered and bound on start.

Agent stdout → Strip ANSI + escapes → 23 regexes, in-process, no model call

Fast path — destructive

12 patterns. `rm -rf`, `DROP TABLE`, `git push --force`, `git reset --hard`, `kubectl delete`

gate raised immediately

Instant by design: a destructive command must not wait on a network round trip.

Fast path — prompts

11 patterns. `[y/N]`, `(yes/no)`, aider's `(Y)es/(N)o`

wait 400ms for quiet → still a prompt? → known answer → typed

unknown → gate

A `[y/N]` scrolling past is not a prompt waiting for you. Matching eagerly typed into an idle terminal.

Slow path — the Supervisor

Debounce 10s · at most once per agent per 20s · max 2 concurrent

progress question blocker risky_action noise → dropped

Noise is never broadcast. That deletion is the feature.

Gate — the agent is frozen

pendingGate set → terminal brought forward → approve → keystrokes typed reject → Esc, then “stop”

custom → your own text

Resolution has one implementation (`src/gate-resolve.ts`); the REST route and the voice path both call it. While a gate is open, `/org/inject` refuses — a message delivered then would become the answer to that prompt.

Every decision appends to `audit.jsonl`. Approve and reject are equally one click.

Plan — the Supervisor cannot act

Supervisor calls `plan_action` → pending record in `project.json` → you see the exact message

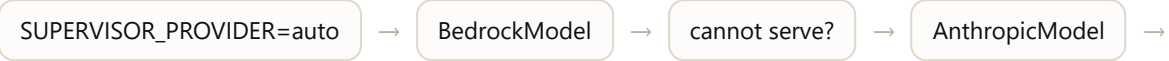
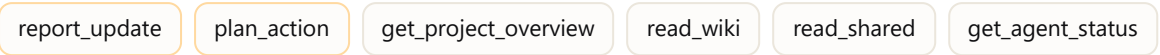
approve → agent:inject reject + reason → nothing sent

`plan_action` is the Supervisor's **only** write tool. It returns *“Plan submitted for human approval”* and sends nothing. The human resolve endpoint is the single delivery path.

Rejected plans are included in later context, so it does not re-propose what you refused.

 AWS Strands Agents

Agent + tool() — two write-adjacent tools and four read-only.



429? step down the ladder

Both paths run through the SDK. Before this, the fallback called the Messages API by hand — so the framework left the running system precisely when the primary provider was down. On a new AWS account, capped near 10k Bedrock tokens/day, that was the common case.

Proving it at runtime

GET /api/health → supervisorHealth

FIELD	MEANS
state	ok · degraded (working, not on the configured provider) · failing · idle · off
provider	which one actually answered
model	the model id it used
strands	whether the last good classification went through the SDK

“The Supervisor works” and “the Supervisor works as a Strands agent” are different claims. Only one of them is the claim this project makes.

Backoff: 1 → 2 → 4 → 8 min, capped at 15, so a dead credential cannot fire one request per batch forever.



awaiting_input and **idle** are ready states, not stuck states. They are exactly when you, the Keeper, or an approved plan send the next instruction. Only a **pendingGate** means blocked — a guard that refused these once made finishing a task equivalent to becoming unreachable.

Claude and Codex report all four. The other runtimes report liveness only, and the UI does not pretend otherwise.

Amazon Nova Sonic

bidirectional

Browser captures PCM16 through an AudioWorklet, gated by a VAD so the socket stays open but audio only flows above the threshold. AWS credentials never leave Node.



Measured **659 ms** end to end: wake, a real tool call, and audio back.

Rejecting is free. Approving is not.

reject — no checks at all

- 1** describe_gate read this exact gate aloud
- 2** that was under **60 seconds** ago
- 3** your own recorded speech says “approve”, un-negated — **a bare “yes” is not enough**
- 4** the gate is still open, text unchanged

Enforced in `src/voice/approval-guard.ts`, **never** by telling the model to be careful. On the pipeline path approving is not disabled — there is **no approve action in the router’s type**, so no transcript can produce one, and the tests assert its absence.

Layout ~/conduit

.env	the only config the packaged desktop app reads
api-keys.json	voice provider keys, never sent to the client
supervisor-log.jsonl	every classification, across all projects
brain-private/	Keeper state, deliberately outside any project cwd
projects/<id>/project.json	project, agents, pending plans
projects/<id>/groupchat.jsonl	one line per message
projects/<id>/audit.jsonl	one line per decision you made
wiki/<name>/	long-term memory — moved on rename
shared_content/<name>/	file handoff — moved on rename

Concurrency

Two processes write `project.json`. Mutation takes a lock with `fs.openSync(file, 'wx')` — atomic create, fails if it exists.

0 lost of 300

valid JSON at every read

never observed empty

Measured with two processes × 150 writes.

The lock deliberately does **not** call `ensureDir`. It did once, and that recreated the directory of a project you had just deleted — resurrecting it on the next write.

`brain-private` sits outside any project cwd because it used to live in the Keeper's own working directory, where the Keeper would read its own history and answer questions about deleted projects.

Protected

- ✓ **CONDUIT_AUTH** — Basic auth over HTTP *and* the WebSocket upgrade, compared in constant time.
- ✓ **Path containment** — filenames never reach `path.join`; `storage.resolveInside` resolves then asserts.
- ✓ **Rendered output** — everything from a model or agent goes through DOMPurify.
- ✓ **Secrets stay in Node** — `/api/voice/config` returns booleans, never keys.

Deliberately **not** protected

The daemon's `/org/*` endpoints are unauthenticated on loopback. Any local process — including an agent Conduit is running — can drive any agent through them, bypassing gates and plan approval. A deliberate trade for a single-user local tool, and the wrong trade on a shared machine. `CONDUIT_AUTH` covers :3200, not this.

Gates catch patterns. They are a seatbelt, not a sandbox. An agent's terminal is a real shell.

No multi-user model. One shared credential, no roles, no per-project permissions.

COMMAND	COVERS	RESULT
npm test	gate patterns, policy, voice routing, approval guard, storage concurrency	292 passed
npm run smoke	REST, WebSocket, PTY dispatch, wiki, MCP routing	61 passed
check:ui	buttons, routes, and every write read back through a different route	clean
check:layout	11 surfaces × 4 widths — overflow, clipping, overlap, contrast, tap targets	no defects
browser-check	real interaction, asserts no uncaught exceptions	clean
check:org	every <code>/org/*</code> endpoint the Keeper uses	clean
check:lifecycle	deleting a project mid-flight, restarting the daemon	clean
check:abuse	traversal, absurd input — no 5xx, no dropped connections	clean
check:agents	one agent of every CLI type	6 / 6
check:multi	four agents at once, gates, group chat, MCP messaging	26 / 26
check:keeper	the Keeper answers with working tools	clean
check:voice-live	wake, tool call, audio back	659 ms
check:desktop	own daemon, all agent types, Bedrock parity, no orphaned processes	31 / 31
check:bedrock	distinguishes IAM, entitlement and quota failures	diagnostic
typecheck	strict TypeScript, client and server	0 errors
No test framework. Each is a plain Node script that prints what it checked and exits non-zero.		